

Theorey Assignment-2: Model Solution

Full Marks: 50

Question 1 *The police department in the city of Computopia has made all the streets one-way. But the mayor of the city still claims that it is possible to legally drive from one intersection to any other intersection.*

- (a) *Formulate this problem as a graph theoretic problem and explain why it can be solved in linear time. (7 Marks)*
- (b) *Suppose it was found that the mayor's claim was wrong. She has now made a weaker claim: "if you start driving from town-hall, navigating one-way streets, then no matter where you reach, there is always a way to drive legally back to the town-hall. Formulate this weaker property as a graph theoretic problem and explain how it can be solved in linear time. (8 Marks)*

Solution: *We give the solutions in part (a) and part (b) separately.*

- (a) **Graph Construction:** *The idea is to construct a directed graph G as follows. Let there be n intersections (including some dead ends if any). We represent every intersection by a vertex v_i . Hence, the vertices are $v_1, \dots, v_n$. As per the road-map or the street-map, if there is a single street between one intersection v_i to another intersection v_j with no other intersection in between, then add an undirected edge. If the police makes the policy such that one can legally drive only from v_i to v_j , then put the directed edge $v_i \rightarrow v_j$. This completes the construction of $G = (V, E)$.*

Rubric:

- **2 Marks** for representing the intersections by vertices and defining how an undirected edge is equivalent to street.
- **1 Marks** for indicating the direction of the street based on how police has made a particular street one-way.

Algorithm: *As per mayor's claim, it is possible to legally drive from one intersection to another intersection if and only if for every pair of vertices u and v , there is a path from u to v as well as from v to u . Hence, the algorithm must check if the graph is strongly connected.*

1 Marks for mentioning the condition – "Mayor's claim is correct if and only if the directed graph is strongly connected" (or equivalently, the directed graph has only one strongly connected component).

3 Marks for mentioning the algorithm.

- Compute the strongly connected components of G (no need to elaborate the algorithm).

- If G has only one strongly connected component, then output that mayor's claim is correct. Otherwise, mayor's claim is wrong.

(b) The graph formulation is the same as before. The 'town-hall' corresponds to a vertex/node s in the graph G . (2 Marks)

4 marks for algorithm:

- Compute the strongly connected components $C_1, \dots, C_r$ of G starting from s (the vertex corresponding to town-hall).
- Then, for every vertex $u \in V(G)$, store an array $\text{Belong}[u]$ to indicate the index i such that $u \in C_i$.
- Let $s \in C_1$ (without loss of generality).
- For every vertex $u \in C_1$ and for each of the edges $u \rightarrow v$ (i.e. the vertices that are adjacent to u but the edges are directed from u to that vertex v adjacent to u), check if $v \notin C_1$ (i.e. $\text{Belong}[v] \neq 1$). If $v \notin C_1$ for some v , then output that mayor's weaker claim is incorrect. Otherwise, output that mayor's claim is correct.

2 Marks for Running time: It is sufficient if they mention that the information on $\text{Belong}[u]$ can be stored in $O(|V| + |E|)$ -time and that is how the step 4 (the 4th bullet point) can be executed in $O(|V| + |E|)$ -time. If they just mention that a careful use of union-find data structure can be used to compute $\text{Belong}[u]$ for all vertices without the details, then that is also fine.

Question 2 Given an edge-weighted connected undirected graph $G = (V, E)$ with $n + 20$ edges. Design an algorithm that runs in $O(n)$ -time and outputs an edge with smallest weight contained in a cycle of G . You must give a justification why your algorithm works correctly. (15 Marks)
(Hint: Formulation into graph theoretic problem is not necessary as the graph is given already.)

Solution: This problem-solving needs to use the ideas that were taught in the tutorial and/or lecture. Here are a few things to observe.

If we output edge e , then we have to ensure that the e is part of some cycle in G . Any edge which is not part of any cycle of G is a bridge. Specifically, a bridge is an edge if and only if its removal disconnects the graph. The essential idea is to find all the bridges of G . Then the remaining edges of G will be part of cycle(s). The minimum weight edge among these edges will be the desired output. Once we detect the bridges, the rest of the process will take $O(|V| + |E|)$ which is $O(n)$ in this case.

Bridges can be found in many different ways. In some cases finding all the bridges explicitly may not be necessary. However, any correct algorithm should identify all of them and discard them from the possibilities. Here, I am going to show the modified DFS algorithm which uses a similar idea of the cut vertex (done in the tutorial).

In a DFS tree, an edge (u, v) (u is the parent of v in a DFS tree) is a bridge if and only if

1. There does not exist any other alternative to reach v from u .

2. Or none of the vertices v and its descendants in the DFS traversal tree has a back-edge to vertex u or any of its ancestors. The reason is that each back edge gives us a cycle, and no edge that is a member of a cycle can be a bridge.

Approach 1

We will be maintaining two arrays:

- $in[]$ - this stores the time when the vertex is first visited.
 - $low[]$ - for every vertex u it stores the discovery time of the earliest visited vertex to which the vertex u or any of its descendants have a back edge. Thus, $low[u]$ is the minimum of three values,
 - $in[u]$: the discovery time of vertex u .
 - $in[p]$: where p is an ancestor of u in the DFS tree and have an edge with p , i.e there is a back edge between p and u .
 - $low[v]$: where there is a tree edge between u and v . This means that v is a vertex directly connected to u with an edge and v is not the parent node of u in the DFS tree.
1. Initially, we will initialize $low[]$ to -1. We will also initialize one variable $timer = 0$ to keep track of the current discovery time.
 2. Now, there is a back edge from v or one of its descendants to ancestors of u (u is the parent of v) if and only if $low[v] \leq in[u]$. This is due to the fact that if there is a back edge then the value of $low[v]$ must be lesser than $in[u]$. If $low[v] = in[u]$ then the back edge comes directly from v to u . Thus, the edge (u, v) in the DFS tree is a bridge if and only if $low[v] > in[u]$. Updating the $low[]$ for all the vertices can be done in $O[|V| + |E|]$ time during the DFS traversal. The approach is exactly similar to the cut vertex done in the tutorial.
 3. Using this approach all the bridges of the graph can be found in $O(n)$ -time since the graph has only $n + 20$ edges.
 4. Since a bridge cannot participate in any cycle, the other edges that are bridges do participate in one of the cycles.
 5. Among all the other edges that are not bridge, find out an edge of G with the smallest weight.

Approach 2: This is more elaborate (although essentially uses ideas of the bridge edge finding algorithm)

Algorithm 1 detects the minimum weight edge of one cycle. As soon as a back edge (vu) is detected (Line 2), the algorithm traces back the parent pointers all the way to u and finds the minimum weight edge. This is accomplished in Lines 2-8. The rest is just DFS.

Algorithm 2 calls Algorithm 1. Once it finds the minimum weight edge of one cycle, it deletes that edge from the graph and continues the process from scratch. The process is repeated 16 times since after that there won't be any more cycles.

Algorithm 1: Find-Min-Edge-One-Cyle

Data: Graph $G = (V, E)$ as adjacency list, weights $w(e), \forall e \in E$, a source vertex v

```
1 Mark  $v$  as visited
2 for all  $u$  in adjacency list of  $v$  do
3   if  $u$  is already visited then
4      $\text{min\_edge} = w(vu)$ 
5      $v' = v$ 
6     while  $v' \neq u$  do
7        $\text{min\_edge} = \min\{\text{min\_edge}, w(v', \text{parent}[v'])\}$ 
8        $v' = \text{parent}[v']$ 
9     return  $\text{min\_edge}$  and abort recursion (abort is not precise but just for
      pseudocode)
10  else
11     $\text{parent}[u] = v$ 
12    Find-Min-Edge-One-Cyle( $G, w, u$ ) (usual DFS)
```

Algorithm 2: Find-Min-Edge-All-Cycles

Data: Graph $G = (V, E)$ as adjacency list, weights $w(e), \forall e \in E$

```
1 Mark  $v$  as unvisited for all vertices  $v$ 
2 for  $i=1$  to 16 do
3   Mark  $v$  as unvisited for all vertices  $v$ 
4   Mark  $\text{parent}[v]$  as NULL for all vertices except  $s$  (some arbitrary source)
5   Initialize  $\text{cur\_min} = \infty$ 
6   Let  $(uv) = \text{Find-Min-Edge-One-Cycle}(G, w, s)$ 
7    $\text{cur\_min} = \min\{\text{cur\_min}, w(uv)\}$ 
8 return  $\text{cur\_min}$ 
```

Rubric:

- +8 for the algorithm/pseudocode: (5/3/0) for correct/partially-correct/incorrect logic and output; (2/0) for clear/unclear steps and explanation; (1/0) handling all cases/minor mistakes.
- +3 Justification of correctness: The property/observation of the bridge/cycle edge which leads to the algorithm has to be written. It is significant to argue that bridges are not considered in their algorithm and the rest of all the edges are considered.
- +4 Running time: (2/0) For a step-wise justification of the running time of the algorithm to be. (2/0) Observation on the running time to be $O(n)$.

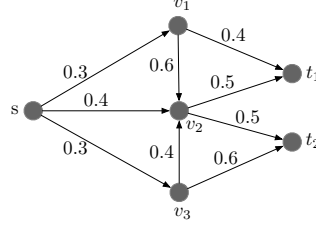


Figure 1: An illustration of Question 3. The probabilities for each of the directed edges are given. The probability that a random walk uses the path $s \rightarrow v_1 \rightarrow t$, then the probability that the walk reaches from s to t using this path is $0.3 \times 0.4 = 0.12$. Similarly, for the path $s \rightarrow v_1 \rightarrow v_2 \rightarrow t$, the probability that the random walk reaches from s to t using this path is $0.3 \times 0.6 \times 0.5 = 0.09$.

Question 3 Suppose that G be a directed acyclic graph with following features.

- G has a single source s and several sinks $t_1, \dots, t_k$.
- Each edge $(v \rightarrow w)$ (i.e. an edge directed from v to w) has an associated weight $p(v \rightarrow w)$ between 0 and 1.
- For each non-sink vertex v , the total weight of all the edges leaving v is $\sum_{(v \rightarrow w) \in E} Pr(v \rightarrow w) = 1$.

The weights $Pr(v \rightarrow w)$ define a random walk in G from the source s to some sink t_i ; after reaching any non-sink vertex v , the walk follows the edge $v \rightarrow w$ with probability $Pr(v \rightarrow w)$. All the probabilities are mutually independent. Describe and analyze an algorithm to compute the probability that this random walk reaches sink t_i for every $i \in \{1, \dots, k\}$. You can assume that an arithmetic operation takes $O(1)$ -time. (see Figure 1 for an illustration) **(20 Marks)**

Solution: What is crucial is that a random walk from s reaches a sink t_i , if that random walk avails one of the directed paths from s to t_i . We treat each of them as events. Define “event E_i happens” if a random walk from s reaches t_i . Observe that for two distinct i, j , if E_i happens then E_j cannot happen implying that they are mutually exclusive to each other. Hence $Pr[E_i \cap E_j] = 0$ and $Pr[E_i \cup E_j] = Pr[E_i] + Pr[E_j]$.

Rubric: 1 Marks for mentioning this mutually exclusiveness property.

Furthermore, consider two different paths P_1 and P_2 from s to t_i . If a random walk takes the path P_1 , then also the random walk cannot take the path P_2 . Hence, the probability of reaching s to t_i is taking one of the directed paths from s to t_i . If P_1 and P_2 are the only two distinct paths from s to t_i , then probability for the random walk reaching t_i is the sum of the probability of random walk using P_1 and the probability of random walk using path P_2 . This holds true if there are 3 or more distinct paths from s to t_i in the graph.

Rubric: 1 Marks for mentioning this above condition as well.

Also, by definition, if a random walk uses edge e_1 and e_2 are mutually independent. It means that If path P from s to t_i uses $e_1, \dots, e_j$, then the probability that the random walk uses path P is

$\prod_{k=1}^j Pr[e_j]$. Note that $Pr[e_j] = Pr(v \rightarrow w)$ if $e_j = v \rightarrow w$ the directed edge.

Rubric: 1 Mark for mentioning this.

Moral of the story: Use dynamic programming.

Preprocessing Stage: Construct topological sort ordering $(v_1, \dots, v_n)$ of G . **(2 Marks)** Every edge has the property that $v_i \rightarrow v_j$ with $i < j$.

It is okay if they (implicitly) assume that $s = v_1$ and all other vertices can be reached from s .

Subproblem Definition: $VAL(v_i)$ is the probability that a random walk reaches from $s (= v_1)$ to v_i . **(2 Marks)**

Recurrence Relation: $VAL(s) = 1 = VAL(v_1)$ (base case).

For all $j > 1$, $VAL(v_j) = \sum_{i < j: (v_i \rightarrow v_j) \in E} \left(VAL(v_i) Pr(v_i \rightarrow v_j) \right)$.

Rubric: 1 Mark for the base case and **2 Marks** for the other part of the recurrence.

Subproblems that solve the actual problem: The final solution is $VAL(t_1), \dots, VAL(t_k)$. Of course the t_k corresponds to some $v_{f(k)}$. **(2 marks)**

Algorithm Description: The algorithm works as follows.

- Construct a topological sort ordering $(v_1, \dots, v_n)$ and suppose $s = v_1$.
- We store the values in an array $TABLE[i]$ such that $TABLE[i] = VAL(v_i)$.
- Initialize $TABLE[1] = VAL(s) = 1$.
- For every $j = 2, 3, \dots, n$
set

$$TABLE[j] = \sum_{i < j: (v_i \rightarrow v_j) \in E} Pr(v_i \rightarrow v_j) TABLE[i]$$

- For every $i = 1, \dots, k$ let $v_{f(k)} = t_k$.
- For every $i = 1, \dots, k$, output $TABLE[f(i)]$.

Rubric: (5 Marks) If somebody uses memoization approach and that is equivalent to this, then also he gets the same credit.

In case of a partially correct answer, deduct 2 marks.

Running time explaiantion: (3 Marks) Computing $TABLE[j]$ requires an approach to go through the adjacency list of the graph. Hence, computing $TABLE(j)$ requires checking $Table(i)$ values for every $v_i \rightarrow v_j$ edge. Hence the total running time is $O(|V| + |E|)$.